

Contents

Part 1 – IaaS Deployment using EC2	2
Objective	2
Steps Followed	2
Part 2 – PaaS Deployment using Elastic Beanstalk + S3	6
Objective	6
Steps Followed	6

Part 1 – IaaS Deployment using EC2

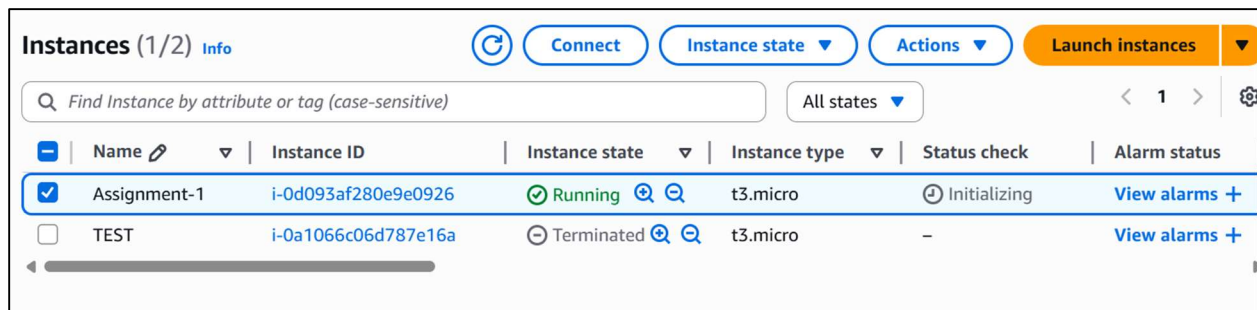
Objective

To deploy a simple Flask web application connected to a MySQL database using AWS EC2 (Infrastructure as a Service).

Steps Followed

1. Launched an EC2 Instance

- Instance type: t3.micro
- OS: Ubuntu 24.04 LTS
- Key pair: assignment1-key.pem
- Allowed inbound rules: HTTP (80), HTTPS (443), and Custom TCP 5000.



2. Connected to the Instance

- Used *EC2 Instance Connect* from AWS Console.

```
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@ip-172-31-30-130:~$
```

3. Installed Docker and Created Flask App Directory

```
ubuntu@ip-172-31-30-130:~$ groups
ubuntu adm cdrom sudo dip lxd docker
ubuntu@ip-172-31-30-130:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-30-130:~$ mkdir flask_mysql_app
ubuntu@ip-172-31-30-130:~$ cd flask_mysql_app
ubuntu@ip-172-31-30-130:~/flask_mysql_app$ nano app.py
ubuntu@ip-172-31-30-130:~/flask_mysql_app$
```

4. Added Files

- app.py

```
GNU nano 7.2 app.py *
from flask import Flask, request, jsonify
import mysql.connector
import os

app = Flask(__name__)

# Database connection details (from environment variables)
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_USER = os.getenv("DB_USER", "root")
DB_PASS = os.getenv("DB_PASS", "password")
DB_NAME = os.getenv("DB_NAME", "flaskdb")

@app.route('/')
def index():
    return "<h2>Flask + MySQL on Docker (EC2)</h2>"

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Locatio
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To I
```

- requirements.txt

```
GNU nano 7.2
flask
mysql-connector-python

```

- Dockerfile

```
GNU nano 7.2
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

- docker-compose.yml

```

GNU nano 7.2                                     docke
version: '3.8'

services:
  db:
    image: mysql:8.0
    container_name: mysql_c
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: flaskdb
      MYSQL_USER: flaskuser
      MYSQL_PASSWORD: flaskpass
    volumes:
      - db_data:/var/lib/mysql
    ports:

```

5. Built and Ran Containers

```

ubuntu@ip-172-31-30-130:~/flask_mysql_app$ nano docker-compose.yml
ubuntu@ip-172-31-30-130:~/flask_mysql_app$ docker-compose down
Removing mysql_c ... done
Removing network flask_mysql_app_default
ubuntu@ip-172-31-30-130:~/flask_mysql_app$ docker-compose up -d
Creating network "flask_mysql_app_default" with the default driver
Creating mysql_c ... done
Creating flask_c ... done
ubuntu@ip-172-31-30-130:~/flask_mysql_app$

```

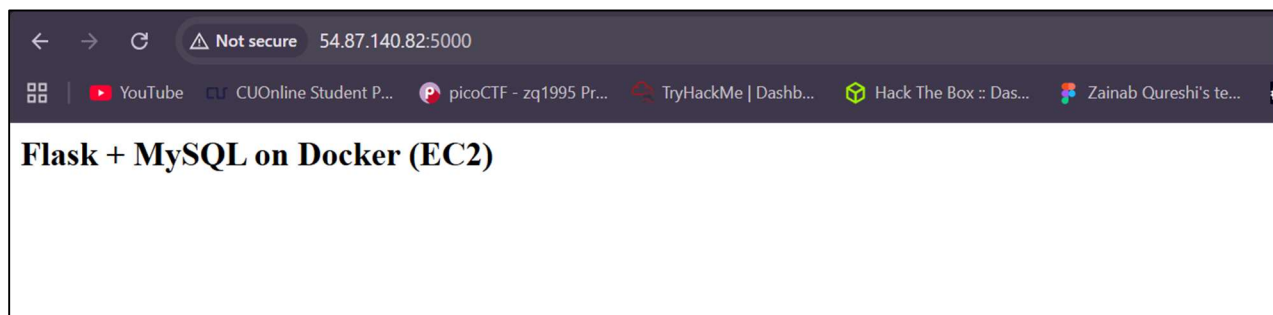
```

ubuntu@ip-172-31-22-157:~/flask_mysql_app$ docker-compose ps
  Name                    Command                                State      Ports
-----
flask_c                  python app.py                          Up         0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
mysql_c                  docker-entrypoint.sh mysqld           Up         3306/tcp, 33060/tcp
ubuntu@ip-172-31-22-157:~/flask_mysql_app$ curl http://127.0.0.1:5000
<h2>Flask + MySQL on Docker (EC2)</h2>ubuntu@ip-172-31-22-157:~/flask_mysql_app$

```

6. Verified Application

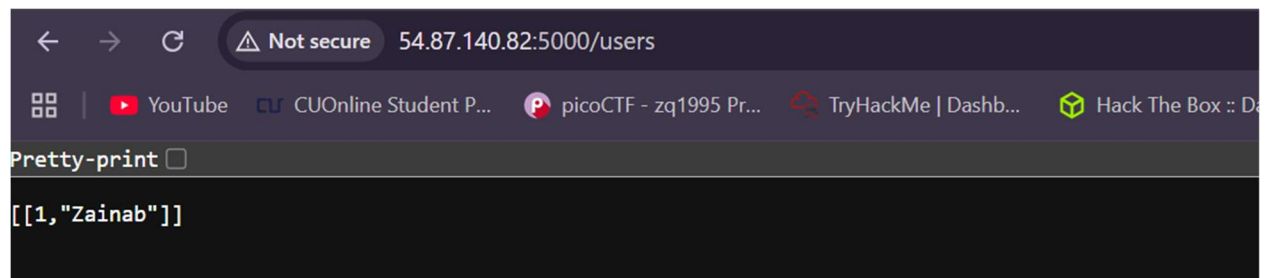
- Opened the app in browser:
<http://54.87.140.82:5000>



7. Verify Database Connectivity

- Optional MySQL check inside container:
- Create sample table:
- Now visiting <http://54.87.140.82:5000/users> returns the JSON data from MySQL.

```
ubuntu@ip-172-31-22-157:~/flask_mysql_app$ curl http://127.0.0.1:5000
<h2>Flask + MySQL on Docker (EC2)</h2>ubuntdocker-compose exec db mysql -uroot -ppassword -e "USE flaskdb; CREATE TABLE IF NOT EXISTS users
(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100));MENT PRIMARY KEY, name VARCHAR(100));"
mysql: [Warning] Using a password on the command line interface can be insecure.
ubuntu@ip-172-31-22-157:~/flask_mysql_app$ curl -X POST -H "Content-Type: application/json" -d '{"name":"Zainab"}' http://127.0.0.1:5000/add
{"message":"Zainab added!"}
ubuntu@ip-172-31-22-157:~/flask_mysql_app$ curl http://127.0.0.1:5000/users
[[1,"Zainab"]]
ubuntu@ip-172-31-22-157:~/flask_mysql_app$
```



8. IaaS Characteristics

- This part demonstrates the **Infrastructure as a Service (IaaS)** model because AWS provided the virtual infrastructure (EC2, storage, and networking), while I configured the operating system, installed Docker, and deployed both the Flask web application and MySQL database, taking full control of the software stack above the infrastructure layer.

Part 2 – PaaS Deployment using Elastic Beanstalk + S3

Objective

To deploy the same Flask application using AWS PaaS model via **Elastic Beanstalk**, storing code in S3.

Steps Followed

1. Prepared App Folder

```
flaskapp/  
├── app.py  
├── requirements.txt  
├── Procfile  
└── .ebextensions/  
    └── python.config
```

- App.py

```
from flask import Flask  
application = Flask(__name__)  
  
@application.route('/')  
def home():  
    return "Hello from Zainab Qureshi's Flask App on Elastic Beanstalk!"  
  
if __name__ == "__main__":  
    application.run(host="0.0.0.0", port=8000)
```

- Requirements.txt

```
Flask==3.1.0  
gunicorn==23.0.0
```

- Procfile

```
web: gunicorn app:application --bind 0.0.0.0:8000
```

- Python.config

```
option_settings:  
  aws:elasticbeanstalk:container:python:  
    WSGIPath: app:application
```

2. Compressed Folder

- Zipped the entire flaskapp directory → flaskapp.zip.

3. Uploaded Code to S3

- Created bucket: flask-web-app-zainabqureshi
- Uploaded flaskapp.zip
- Set permissions: Public access (for testing only).

The screenshot shows the 'Create bucket' page in the AWS S3 console. The breadcrumb trail is 'Amazon S3 > Buckets > Create bucket'. The 'AWS Region' is set to 'US East (N. Virginia) us-east-1'. Under 'Bucket type', the 'General purpose' option is selected, with a description: 'Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.' The 'Directory' option is also visible. The 'Bucket name' field contains 'flask-web-app-zainabqureshi'. Below the field, a note states: 'Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn More](#)'. There is a section for 'Copy settings from existing bucket - optional' with a 'Choose bucket' button. At the bottom, the format 's3://bucket/prefix' is shown.

The screenshot shows the details of the 'flaskapp.zip' object in the AWS S3 console. The breadcrumb trail is 'Amazon S3 > Buckets > web-app-zainabqureshi1 > flaskapp.zip'. The object name is 'flaskapp.zip'. Action buttons include 'Copy S3 URI', 'Download', 'Open', and 'Object actions'. The 'Properties' tab is selected. The 'Object overview' section displays the following information:

Owner 9c95924db062ce59903b0b3266dc714b462a63251813782378e94e097838ffd4	S3 URI s3://web-app-zainabqureshi1/flaskapp.zip
AWS Region US East (Ohio) us-east-2	Amazon Resource Name (ARN) arn:aws:s3:::web-app-zainabqureshi1/flaskapp.zip
Last modified October 7, 2025, 14:57:36 (UTC+05:00)	Entity tag (Etag) 72986ebc9eb10d4d429c8a2689293bcb
Size 1.2 KB	Object URL https://web-app-zainabqureshi1.s3.us-east-2.amazonaws.com/flaskapp.zip

4. Created Elastic Beanstalk Environment

- Platform: Python 3.11 on 64bit Amazon Linux 2023.
- Environment type: *Single Instance*.
- Instance type: t3.micro.
- Application code: S3 URL of flaskapp.zip.

Step 1: Configure environment

[Edit](#)

Environment information

Environment tier

Web server environment

Application name

zainabqureshi-web-app

Environment name

Zainabqureshi-web-app-env

Application code

s3://web-app-zainabqureshi1/flask_mysql_app.zip

Platform

arn:aws:elasticbeanstalk:us-east-2::platform/Python 3.11 running on 64bit Amazon Linux 2023/4.7.2

5. Verified Successful Deployment

- Domain: <http://zainabqureshi.us-east-2.elasticbeanstalk.com>
- Health: **OK**
- Application accessible publicly.

Environment successfully launched.

Zainabqureshi-webapp-env-1 [Info](#)

[Actions](#)[Upload and deploy](#)

Environment overview

Health

✓ Ok

Environment ID

e-avmpggctcv

Domain

zainabqureshi.us-east-2.elasticbeanstalk.com

Application name

[Zainabqureshi-webapp](#)

Platform

[Change version](#)**Platform**

Python 3.11 running on 64bit Amazon Linux 2023/4.7.2

Running version

zainabqureshi3

Platform state

✓ Supported

← → ↺ ⚠ Not secure zainabqureshi.us-east-2.elasticbeanstalk.com



YouTube



CUOnline Student P...



picoCTF - zq1995 Pr...



TryHackMe | Da

Hello from Zainab Qureshi's Flask App on Elastic Beanstalk!

6. PaaS Characteristics

- This project demonstrates the **Platform as a Service (PaaS)** model because AWS Elastic Beanstalk automatically handled the provisioning, load balancing, scaling, and health monitoring of the application environment. I only uploaded my Flask application code (stored in an S3 bucket) and specified configuration details such as the Python platform and instance type. AWS managed the underlying infrastructure, runtime environment, and web server (Nginx + Gunicorn), allowing me to focus solely on deploying and maintaining the application logic rather than managing servers or containers.